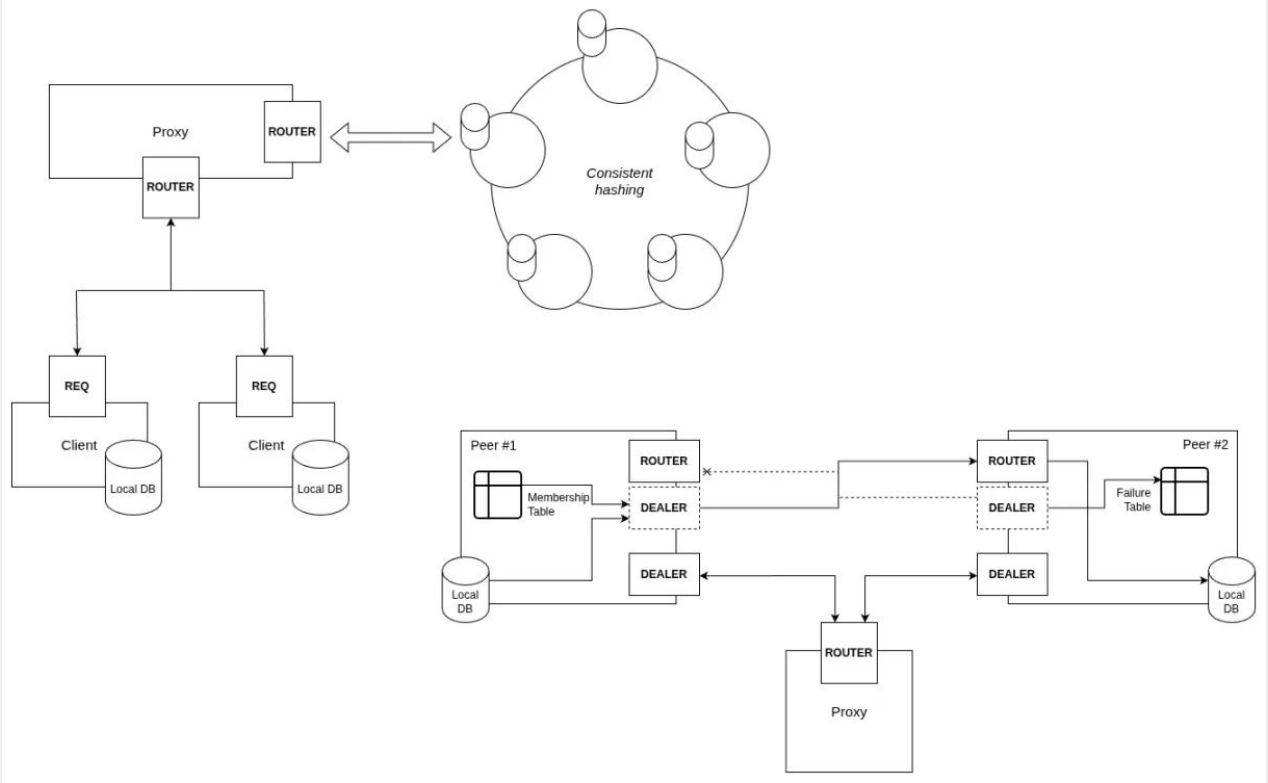
Shopping Lists on the Cloud

Large Scale Distributed Systems
Group 4 | Class 6

Project Overview

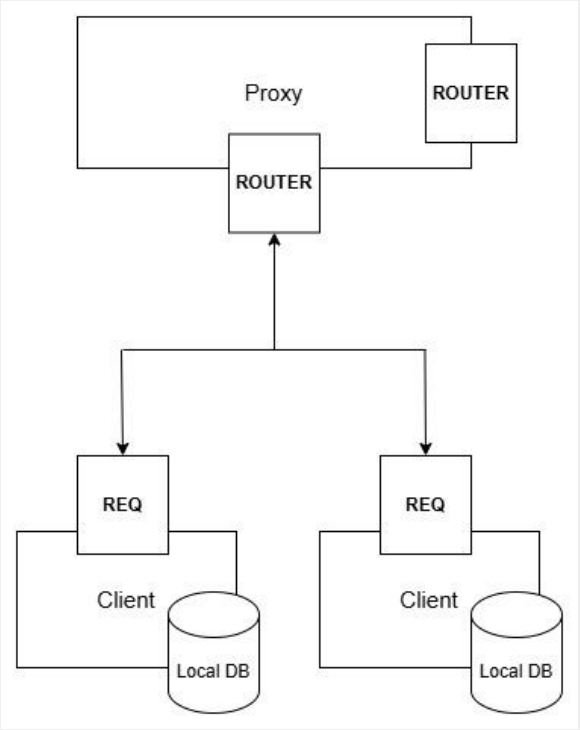
- **Local-First Architecture:** The application is offline-first, persisting data locally on the user's device while synchronizing with the cloud for sharing and backup
- **Shared Shopping Lists:** Each shopping list is identified by a unique ID that can be shared, allowing any user with the ID to add or update items
- **Scalable Cloud Design:** The cloud backend, heavily inspired by the Dynamo architecture, must support millions of users and avoid data access bottlenecks
- **Concurrent Updates:** The application handles concurrent updates through the use of conflict-free replicated data types (CRDTs)

Architecture Diagram



Client

- Maintains local database for offline access
- Sends requests to proxy via REQ socket
- Merges local and remote shopping lists (when possible, normally after operations such as read/write lists or shopping list items)
- Shopping lists can be shared among users



Proxy

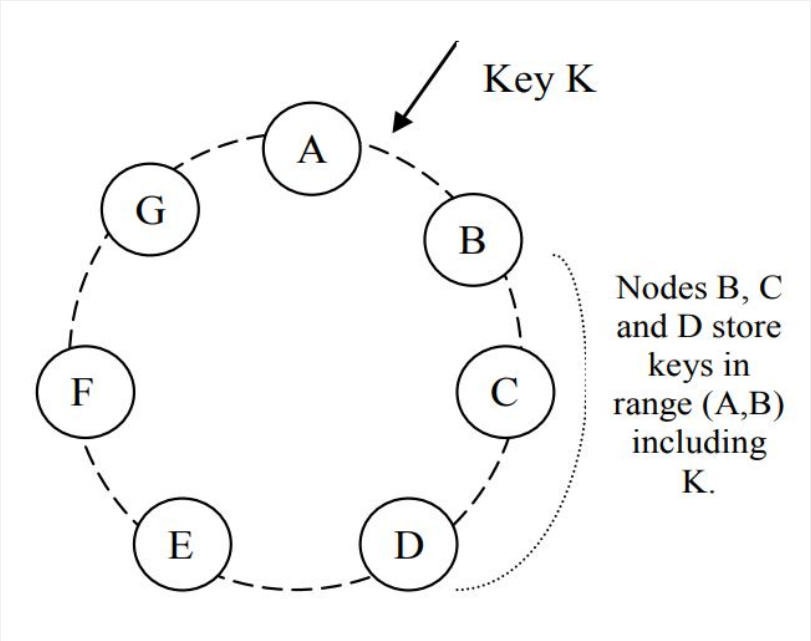
- **Sockets**: ROUTER on frontend and backend
- **Request Routing**: Uses *consistent hashing* (using MD5 hashing algorithm) to assign shopping list IDs to a coordinator server
- **Server Discovery**: Servers register with the proxy by sending a READY message; proxy adds them to the hash ring
- **Message Forwarding**
    - **Client requests**: Proxy prepends the coordinator server's identity and forwards to backend.
    - **Server replies**: Proxy strips server identity and forwards response to the client.

Server Cluster - Distributed Peer-to-peer Architecture

- **Consistent hashing** for data partitioning (using *virtual nodes* alongside *replicas* N)
- **Quorum-based replication** (configurable R/W nodes, ensures R + W > N)
- **Gossip protocol** for membership management
- **Failure Detection** upon unsuccessful write (**shaky*)
- **Hinted Handoff** for temporary failures (**shaky*)

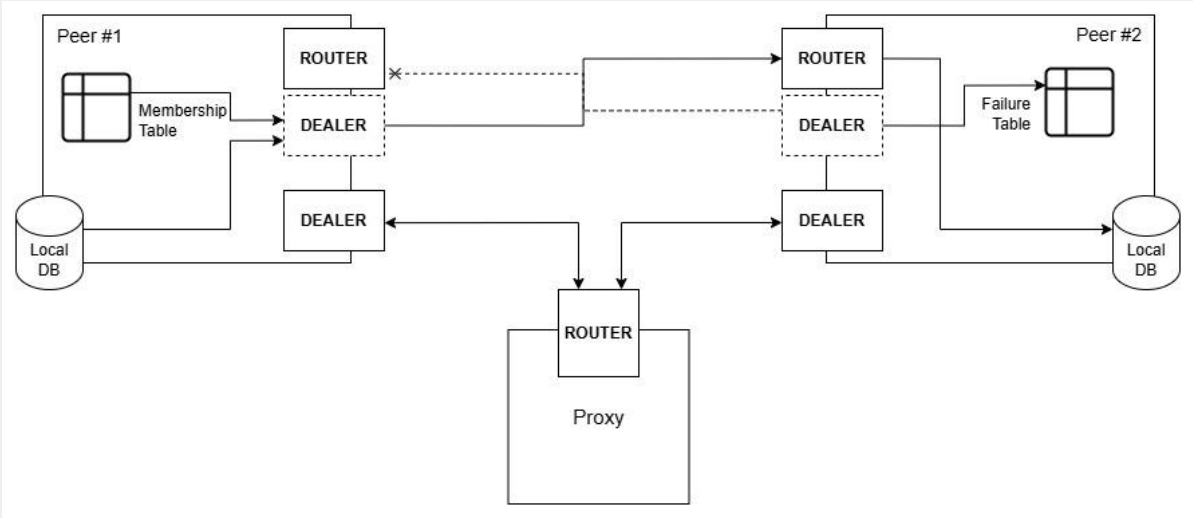
Consistent Hashing

- Three positions on the hash ring are assigned to each server
- The coordinator is determined using the shopping list IDs
- Each list is stored on three consecutive replicas



Gossip Protocol

- Every second, each server picks a random peer and sends its complete membership table (UUID –> address mapping)
- Receiving peer merges the table with its own *local* knowledge



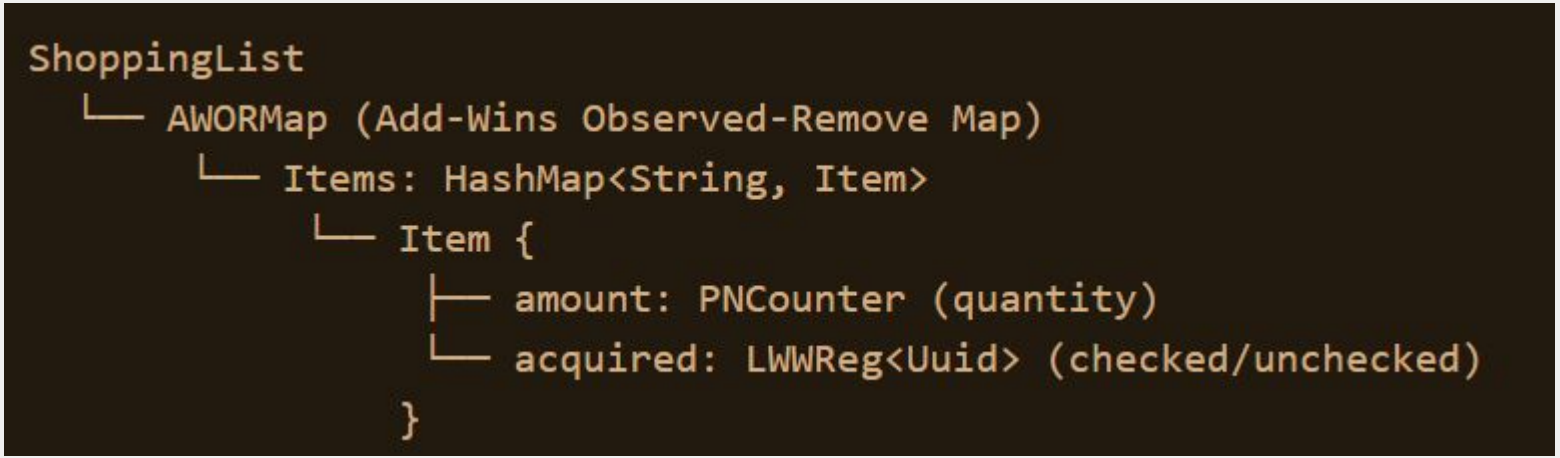
Hinted Handoff - Handling Temporary Failures

Coordinator tries to replicate → Node A is down → Find next available node B in hash ring → Send data to Node B with hint “this is for Node A”

- **Temporary storage**: Node B stores the data tagged the with original destination UUID
- **Periodic Replay**: Check handoff storage, if no pending handoffs, try sending to original node. If successful, delete from handoff storage

Conflict-Free Replicated Data Types (CRDTs)

Enable concurrent edits from multiple users without coordination, while guaranteeing convergence.



PNCounter - Item Quantity

- **Two Grow-Only Counters**: **P** (positive) counter for increments and **N** (negative) counter for decrements
- The value is **P** - **N,** always mathematically correct
- It’s commutative and idempotent, meaning the merge order doesn't matter and can merge multiple times safely
- **On Merge**:
    - Merge P and N counters independently
    - take max() per actor in each counter

```rust
pub struct PNCounter {
    pub p: GCounter,
    pub n: GCounter,
}
```

LWWReg - Acquired Flag

- **Last-Writer-Wins Register** is used for the acquired flag in shopping list items
- Stores a value, a logical clock and an actor ID (representing who performed the operation)
- **Conflict resolution**: On merge, keeps the value with the highest clock; if clocks are equal, breaks ties by actor ID.

```rust
pub struct LWWReg<A> {
    pub val: u32,
    pub clock: u32,
    pub actor: A,
}
```

AWORMap - Shopping List

- Maps Item names to Item objects
- **Add-wins semantics**: If add and remove operations conflict, the addition takes precedence.
- **Merge Resolution**:
    - Union of both maps
    - Items with same key merge recursively

```rust
pub struct AWORMap {
    pub items: HashMap<String, Item>,
}
```

Conclusion - The Not So Good, the Bad and the Ugly…

- Our current implementation is missing some key details in order to make the solution *truly* **reliable** → *Hinted Handoff *should've** been a simple mechanism but maybe adding it one hour before the delivery is not that good of an idea…
- *Failure detection* is not being done properly, although our implementation follows *Amazon Dynamo's* approach (1 request failed to send to peer = mark peer as **failed**), peers almost never detect failure. This could be related to ZeroMQ sockets and how we handled `socket.send()` and `socket.connect()` error detection.
- To maintain a low number of opened sockets, we currently continuously open/close sockets to talk to a peer. This is obviously an **awful** approach since it adds a massive overhead to performance and, since there is now way to fully know if a message arrived to its destination, if a socket is closed to soon, the message may never reach its destination.

Conclusion - The Not So Good, the Bad and the Ugly…

- New servers/peers adding and leaving the cluster functionalities are **semi-implemented**. Although on server startup we're able to add any X nodes to the ring, at run time the cluster would **block** (*intuition* = probably *deadlock* somewhere…). Leaving the cluster is **good enough** for testing purposes, since it only removes a peer from the *hash ring* though gossip and so never reaches the proxy's own *hash ring*.
- Graceful shutdown is basically not implemented, this would require passing a token with `tokio` to every spawned *task* (similar to *thread*). This causes some very weird behaviour when adding/removing nodes, basically it is impossible for a leaving node to fully shutdown since only its main thread is being shutdown (keeping its *gossip*, *failure detection*, …)

Conclusion - The Not So Good, the Bad and the Ugly…

- *Permanent replica failure* is also **not accounted** for and so no periodic replication across peers is done (Merkle trees sounds hard, probably could've achieved similar with CRDTs).
- Overall, we believe this project had **a lot of potential** but is unfortunately missing some key details in order to truly achieve *Dynamo-like* performance, availability and scalability but, in the end, we at least learned Rust!!! 🥲